Studio Nx

JS 314 — 시퀀서 로딩 화면

2026-10-21

JavaScript 314 — 시퀀서 로딩 화면 (`pages/sequencer-loader.js`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-loader.js`, `studionx/frontend/studioNx.html`의 `#sq-loader`, `studionx/frontend/static/css/pages/studio.css`의 `.sq-ld` 규칙
작업	S1-14 시퀀서 로딩 화면: 화면을 열 때 덮는 로딩 화면의 계산, 그리기, 열기 (목업 `docs/page_mockups/sequencer/04~07`, B "Signal in")
작성	Claude (AI), 2026-10-21, 사용자 검토
이전, 다음	이전 313 화면 오류 수집, 다음 315 로딩 화면 연결과 샘플 미리 받기

1. 하는 일

시퀀서(`studioNx.html`)를 열면 화면 전체를 덮는 로딩 화면이 먼저 나옵니다. 진행률만큼 왼쪽부터 파형이 그려지고, 100%가 되면 `SIGNAL IN`이 켜진 뒤 클릭 없이 신호선을 따라 위아래로 열립니다. 이 파일은 그 로딩 화면의 계산과 그리기를 맡습니다. 무엇을 얼마나 불러왔는지 알려 주는 쪽은 315입니다.

```
studioNx.html 의 #sq-loader (첫 그림부터 화면을 덮음, 마크업은 처음부터 들어 있음)
  createLoader({ mode })        로딩 화면을 붙잡고 진행률 그리기를 시작
    setProject / setStage / setSampleTotal / setSamples     315 가 불러온 만큼 알려 줌
    프레임마다 tick: 실제 진행률 → easeDisplayed 로 부드럽게 따라가기 → 파형, 퍼센트, 켜진 단계 그리기
    그려진 값이 100% 에 닿으면 SIGNAL IN → 0.28초 뒤 열기 → 0.42초 뒤 DOM 에서 치움
    finish()                      모든 단계를 끝으로 보고, 치워지면 풀리는 약속을 돌려줌
```

로딩 화면은 세 가지입니다.

종류	언제	제목 위 글자	세 단계	최소 표시 시간
`open`	`?projectId=`만 있음 (기존 프로젝트)	`OPENING PROJECT`	`PROJECT DATA`, `SAMPLES n/m`, `AUDIO ENGINE`	1.6초
`new`	`?projectId=...&new=1` (마이프로젝트에서 새로 만든 직후)	`NEW PROJECT`	`CREATING PROJECT`, `MASTER TRACK`, `AUDIO ENGINE`	1.1초
`demo`	`projectId` 없음 (예시 화면)	`EXAMPLE SCREEN`	`EXAMPLE PROJECT`, `TRACKS`, `AUDIO ENGINE`	0.8초

2. 언제 쓰나

쓰는 곳	하는 일
`sequencer.js`의 `init` (315)	`loaderMode`로 종류를 정하고 `createLoader`로 붙잡습니다.
`sequencer.js`의 `openWithLoader` (315)	프로젝트 정보와 단계 진행을 알려 주고 `finish`를 기다립니다.
시험 `loader_unit.mjs`, `ui_loader.mjs`	순수 함수와 실제 화면의 진행을 확인합니다.

3. 코드

3.1 순수 함수 (화면 없이 시험)

```
export function loaderMode(search, hasProject) {
  if (!hasProject) return 'demo';
  return new URLSearchParams(search || '').get('new') === '1' ? 'new' : 'open';
}
```

```
export function minDisplayMs(mode, reducedMotion) {
  if (reducedMotion) return 0;
  if (mode === 'open') return 1600;
  if (mode === 'new') return 1100;
  return 800;
}
```

```
export function stagePlan(mode, sampleTotal) {
  if (mode === 'open') {
    const none = sampleTotal === 0;
    return [
      { id: 'data', label: 'PROJECT DATA', weight: none ? 0.55 : 0.3 },
      { id: 'content', label: none ? 'NO SAMPLES' : 'SAMPLES', weight: none ? 0.1 : 0.55 },
      { id: 'engine', label: 'AUDIO ENGINE', weight: none ? 0.35 : 0.15 },
    ];
  }
  if (mode === 'new') {
    return [
      { id: 'data', label: 'CREATING PROJECT', weight: 0.5 },
      { id: 'content', label: 'MASTER TRACK', weight: 0.3 },
      { id: 'engine', label: 'AUDIO ENGINE', weight: 0.2 },
    ];
  }
  return [
    { id: 'data', label: 'EXAMPLE PROJECT', weight: 0.4 },
    { id: 'content', label: 'TRACKS', weight: 0.4 },
    { id: 'engine', label: 'AUDIO ENGINE', weight: 0.2 },
  ];
}
```

```
export function overallProgress(stages) {
  let sum = 0;
  let total = 0;
  (stages || []).forEach(function (s) {
    const w = Number(s.weight) > 0 ? Number(s.weight) : 0;
    const f = Math.max(0, Math.min(1, Number(s.fraction) || 0));
    sum += w * f;
    total += w;
  });
  return total > 0 ? Math.max(0, Math.min(1, sum / total)) : 0;
}
```

```
export function easeDisplayed(displayed, target, elapsedMs, minMs, dtMs) {
  const cap = minMs > 0 ? Math.min(1, Math.max(0, elapsedMs) / minMs) : 1;
  const goal = Math.min(target, cap);
```

```
  if (goal <= displayed) return displayed;
  const next = displayed + (goal - displayed) * (1 - Math.exp(-Math.max(0, dtMs) / 140));
  return goal - next < 0.0005 ? goal : next;
}
```

```
export function wavePath(width, calm) {
  const w = Math.max(60, Math.round(width));
  const mid = WAVE_HEIGHT / 2;
  let d = 'M0 ' + mid;
  for (let x = 0; x <= w; x += 6) {
    const env = Math.pow(Math.sin(Math.PI * x / w), 1.4);
    const y = mid - calm * env * (34 * Math.sin(x * 0.09) * Math.sin(x * 0.013 + 1) + 14 * Math.sin(x *
0.31));
    d += ' L' + x + ' ' + y.toFixed(1);
  }
  return d;
}
```

```
export function projectInfoText(project, trackCount, mode) {
  const parts = [];
  if (project) {
    const bpm = Number(project.bpm);
    if (isFinite(bpm) && bpm > 0) parts.push((Math.round(bpm * 100) % 100 === 0 ? String(Math.round(bp
m)) : bpm.toFixed(2)) + ' BPM');
    if (project.time_sig_num && project.time_sig_den) parts.push(project.time_sig_num + '/' + project.ti
me_sig_den);
    if (project.music_key) parts.push(String(project.music_key + (project.scale_mode ? ' ' + project.sca
le_mode : '')).toUpperCase());
  }
  if (mode === 'new') parts.push('MASTER TRACK ONLY');
  else if (trackCount >= 0) parts.push(trackCount + (trackCount === 1 ? ' TRACK' : ' TRACKS'));
  return parts.join(' · ');
}
```

```
export function collectSampleIds(clips, samplerIds) {
  const out = [];
  const add = function (id) {
    if (id == null || id === '') return;
    const n = Number(id);
    if (isFinite(n) && out.indexOf(n) < 0) out.push(n);
  };
  (clips || []).forEach(function (c) { add(c && c.sample_id); });
  (samplerIds || []).forEach(add);
  return out;
}
```

```
export function sampleCountText(done, total) {
  return Math.max(0, done) + '/' + Math.max(0, total);
}
```

3.2 화면 (로딩 화면을 붙잡고 그리고 엶)

```
export function createLoader(options) {
  const root = typeof document !== 'undefined' ? document.getElementById('sq-loader') : null;
```

```
  const mode = options && options.mode ? options.mode : 'open';
  if (!root) return inertLoader();
  if (typeof window !== 'undefined' && window.__snxNoLoader) {
    // 시험 도구가 화면 조작을 바로 시작할 수 있게 로딩 화면을 보이지 않게 치움 (docs/handoff/p1-28_tests/l
ib.mjs)
    root.remove();
    return inertLoader();
  }

  const reduced = prefersReducedMotion();
  const minMs = minDisplayMs(mode, reduced);
  const q = function (sel) { return root.querySelector(sel); };
  const els = {
    kind: q('.sq-ld__kind'), name: q('.sq-ld__name'), info: q('.sq-ld__info'), line: q('.sq-ld__line'),
wave: q('.sq-ld__wave'),
    flat: q('.sq-ld__flat'), pct: q('.sq-ld__pct'),
    steps: { data: q('[data-step="data"]'), content: q('[data-step="content"]'), engine: q('[data-step
="engine"]') },
  };

  let plan = stagePlan(mode, null);
  const fraction = { data: 0, content: 0, engine: 0 };
  let sampleTotal = null;   // 아직 모름 (null) → setSampleTotal 로 정해짐
  let sampleDone = 0;
  let displayed = 0;
  let lastTs = 0;
  let startTs = 0;
  let raf = 0;
  let finishing = false;
  let opened = false;
  let failed = false;
  let resolveFinish = null;
  const finished = new Promise(function (resolve) { resolveFinish = resolve; });
  let active = true;
  let hardTimer = 0;

  // 화면 뒤의 조작 막기: 로딩 중에는 Tab 으로 뒤 화면에 들어가지 못하게 (inert) — 끝나면 우리가 막은 것만
되돌림
  const inerted = [];
  try {
    Array.prototype.forEach.call(document.body.children, function (el) {
      if (el !== root && !el.inert) { el.inert = true; inerted.push(el); }
    });
  } catch (e) { /* inert 를 모르는 브라우저는 건너뜀 */ }

  // 종류별 문구
  if (els.kind) els.kind.textContent = mode === 'new' ? 'NEW PROJECT' : (mode === 'demo' ? 'EXAMPLE SCRE
EN' : 'OPENING PROJECT');
  root.classList.add('is-' + mode);
  if (reduced) root.classList.add('is-reduced');

  // 신호선: 화면 너비에 맞춘 viewBox 와 파형 (너비가 달라지면 다시 그림)
  function drawLine() {
    if (!els.line || !els.wave) return;
    const w = Math.max(320, Math.round(root.clientWidth || window.innerWidth || 1440));
    els.line.setAttribute('viewBox', '0 0 ' + w + ' ' + WAVE_HEIGHT);
    if (els.flat) { els.flat.setAttribute('x2', String(w)); }
```

```
    els.wave.setAttribute('d', wavePath(w, mode === 'open' ? 1 : 0.55));
  }
  drawLine();
  window.addEventListener('resize', drawLine);

  /** 단계 표시를 계획대로 다시 씀 (샘플 수가 정해지면 문구와 비중이 바뀜) */
  function applyPlan() {
    plan = stagePlan(mode, sampleTotal);
    plan.forEach(function (s) {
      const step = els.steps[s.id];
      if (!step) return;
      const st = step.querySelector('.sq-ld__st');
      const cnt = step.querySelector('.sq-ld__cnt');
      if (st) {
        if (st.firstChild && st.firstChild.nodeType === 3) st.firstChild.nodeValue = s.label + ' ';
        else st.insertBefore(document.createTextNode(s.label + ' '), st.firstChild);
      }
      if (cnt) cnt.textContent = (s.id === 'content' && mode === 'open' && sampleTotal > 0) ? sampleCoun
tText(sampleDone, sampleTotal) : '';
    });
  }
  applyPlan();

  /** 한 프레임 — 목표 진행률로 그려진 값을 옮기고 화면을 고침 */
  function tick(ts) {
    raf = 0;
    if (!active) return;
    if (!startTs) { startTs = ts; lastTs = ts; }
    const dt = ts - lastTs;
    lastTs = ts;
    const target = overallProgress(plan.map(function (s) { return { weight: s.weight, fraction: fraction
[s.id] }; }));
    displayed = reduced ? target : easeDisplayed(displayed, target, ts - startTs, minMs, dt);
    paint();
    if (finishing && displayed >= 1) { signalIn(); return; }
    raf = window.requestAnimationFrame(tick);
  }

  /** 그려진 값을 화면에 반영 — 파형, 퍼센트, 켜진 단계 */
  function paint() {
    if (els.wave) els.wave.style.strokeDashoffset = String(100 - displayed * 100);
    const pct = Math.floor(displayed * 100 + 1e-6);
    if (els.pct) els.pct.textContent = pct + '%';
    root.setAttribute('aria-valuenow', String(pct));
    // 단계는 "그 단계가 끝났고, 파형이 거기까지 그려졌을 때" 켬 (파형보다 앞서 켜지지 않게)
    let acc = 0;
    let total = 0;
    plan.forEach(function (s) { total += s.weight; });
    plan.forEach(function (s) {
      acc += s.weight;
      const step = els.steps[s.id];
      if (step) step.classList.toggle('is-done', fraction[s.id] >= 1 && displayed + 1e-6 >= acc / tota
l);
    });
  }

  /** 100% — SIGNAL IN 을 켜고, 잠깐 뒤 열기 */
```

```
function signalIn() {
  if (opened) return;
  opened = true;
  if (failed) {
    root.classList.add('is-failed');   // NO SIGNAL 을 그대로 두고 연다
  } else {
    root.classList.add('is-in');
  }
  window.setTimeout(open, failed ? 0 : (reduced ? 0 : SIGNAL_HOLD_MS));
}

/** 위아래로 열고 치움 */
function open() {
  root.classList.add('is-opening');
  window.setTimeout(remove, reduced ? 160 : OPEN_MS);
}

function remove() {
  if (!active) return;
  active = false;
  window.clearTimeout(hardTimer);
  window.removeEventListener('resize', drawLine);
  if (raf) window.cancelAnimationFrame(raf);
  inerted.forEach(function (el) { try { el.inert = false; } catch (e) { /* 무시 */ } });
  root.remove();
  resolveFinish();
}

startFrames();
hardTimer = window.setTimeout(function () {
  // 25초가 지나도 안 끝나면 그대로 엶 (끝나지 않은 단계는 켜지 않은 채)
  finishing = true;
  displayed = 1;
  paint();
  signalIn();
}, HARD_TIMEOUT_MS);

function startFrames() { raf = window.requestAnimationFrame(tick); }

return {
  setProject: function (info) {
    if (els.name && info && info.title != null) els.name.textContent = String(info.title);
    if (els.info && info && info.info != null) els.info.textContent = String(info.info);
  },
  setStage: function (id, f) {
    if (fraction[id] === undefined) return;
    fraction[id] = Math.max(fraction[id], Math.max(0, Math.min(1, Number(f) || 0)));   // 줄어들지 않음
  },
  setSampleTotal: function (total) {
    sampleTotal = Math.max(0, Number(total) || 0);
    sampleDone = 0;
    applyPlan();
    if (sampleTotal === 0) fraction.content = 1;
  },
  setSamples: function (done, total) {
    sampleTotal = Math.max(0, Number(total) || 0);
    sampleDone = Math.max(0, Math.min(sampleTotal, Number(done) || 0));
```

```
      applyPlan();
      fraction.content = sampleTotal > 0 ? sampleDone / sampleTotal : 1;
    },
    fail: function () { failed = true; },
    /** 모든 단계를 끝으로 보고, 그려진 값이 100% 에 닿으면 열림. 열려서 치워지면 풀리는 Promise */
    finish: function () {
      if (!finishing) {
        finishing = true;
        fraction.data = fraction.content = fraction.engine = 1;   // 불러오기가 끝났으므로 모든 단계를 끝으로 봄
        if (failed) displayed = 1;                                // 실패했으면 기다리지 않고 바로 엶
      }
      return finished;
    },
    isActive: function () { return active; },
  };
}
```

3.3 마크업과 CSS

마크업은 `studioNx.html`의 `<body>` 첫 요소로 처음부터 들어 있습니다. 그래야 첫 그림부터 화면을 덮고, 아래의 예시 화면이 먼저 비치지 않습니다. 스크립트가 늦게 실행되어도 로딩 화면은 이미 떠 있습니다.

```
<div class="sq-ld" id="sq-loader" role="progressbar" aria-label="Opening the sequencer" aria-valuemin="0" aria-valuemax="100" aria-valuenow="0">
  <div class="sq-ld__half sq-ld__half--top"></div>
  <div class="sq-ld__half sq-ld__half--bot"></div>
  <div class="sq-ld__content">
    ... 로고, 제목(kind, name, info), 신호선(flat, wave), 상태(NO SIGNAL / SIGNAL IN, 퍼센트), 세 단계 ...
  </div>
</div>
```

CSS는 `studio.css` 끝의 `.sq-ld` 규칙입니다. 핵심만 적으면 다음과 같습니다.

규칙	내용
`.sq-ld`	`position: fixed; inset: 0; z-index: 1000`으로 모든 것 위를 덮고, 안전장치 애니메이션 `sq-ld-failsafe`가 30초 뒤에 `visibility: hidden`과 `pointer-events: none`으로 바꿈
`.sq-ld__half`	위와 아래 절반. `.is-opening`이 붙으면 `translateY(-100%)`와 `translateY(100%)`로 열림
`.sq-ld__wave`	`stroke-dasharray: 100`, `stroke-dashoffset: 100`에서 시작. 이 값을 JS가 `100 - 진행률`로 바꿔서 파형이 그려짐
`.is-in`	`NO SIGNAL`이 꺼지고 `SIGNAL IN`이 켜지며 파형이 한 번 빛남
종류가 정해지기 전	`:not(.is-open):not(.is-new):not(.is-demo)`인 동안에는 종류 문구와 세 단계를 숨김 (새 프로젝트인데 `OPENING PROJECT`가 잠깐 비치지 않게)
`prefers-reduced-motion`	위아래로 열지 않고 투명해지며 사라짐

4. 진행률이 움직이는 방식

화면에 그려지는 값(`displayed`)은 실제 진행률(`target`)을 그대로 따라가지 않고 `easeDisplayed`로 부드럽게 따라갑니다. 규칙은 세 가지입니다.

규칙	이유
실제 진행률을 넘지 않고 줄어들지도 않습니다	화면이 거짓말을 하지 않고, 퍼센트가 뒤로 가지 않습니다.
최소 표시 시간(`open` 1.6초)에 비례한 값 이상으로 앞서가지 않습니다	샘플이 없는 프로젝트처럼 순식간에 끝나는 경우에도 파형이 그려지는 모습이 보이게 합니다. 너무 빨리 끝나면 로딩 화면이 깜빡이기만 합니다.
움직임 줄이기 설정이면 최소 시간이 0이고 실제 진행률을 바로 따라갑니다	기다리지 않고, 애니메이션을 보여 주지 않습니다.

세 단계의 점(●)은 "그 단계가 끝났고, 파형이 거기까지 그려졌을 때" 켜집니다. 파형보다 앞서 켜지지 않아서 그림과 글이 어긋나지 않습니다.

5. 최신 문법으로 쓰면

```
// loaderMode 와 overallProgress 를 최신 문법으로 쓴 모양
export const loaderMode = (search = '', hasProject) =>
  !hasProject ? 'demo' : new URLSearchParams(search).get('new') === '1' ? 'new' : 'open';

export const overallProgress = (stages = []) => {
  const total = stages.reduce((s, { weight }) => s + (weight > 0 ? weight : 0), 0);
  const sum = stages.reduce((s, { weight, fraction }) => s + (weight > 0 ? weight : 0) * Math.min(1, Mat
h.max(0, Number(fraction) || 0)), 0);
  return total > 0 ? Math.min(1, Math.max(0, sum / total)) : 0;
};

// 단계 글자 바꾸기
const st = step?.querySelector('.sq-ld__st');
```

전통 문법 (실제 코드)	최신 문법
`new URLSearchParams(search || '')`	기본 매개변수 `search = ''`
`forEach`로 합을 누적	`reduce`
`if (step) { ... }`	옵셔널 체이닝 `step?.querySelector(...)`
`function () { ... }`으로 감싼 콜백	화살표 함수
`Promise`를 `new Promise(...)`로 만들어 `resolve`를 밖에 보관	그대로 (최신 문법으로도 같음)

6. 왜 이렇게

선택	이유
마크업을 `studioNx.html`에 처음부터 넣었습니다	JS가 로딩 화면을 만들어 붙이면 그 순간까지 예시 화면이 먼저 보입니다. 처음부터 덮고 있으면 첫 그림에서부터 로딩 화면이 보입니다 (측정: 첫 표본 23ms에 이미 보임).

선택	이유
30초 안전장치를 CSS에 뒀습니다	JS가 오류로 멈추면 로딩 화면이 영원히 화면을 막습니다. CSS 애니메이션은 JS와 관계없이 흘러서 30초 뒤에 화면을 풀어 줍니다. 시험에서 이 애니메이션이 30초 지연으로 걸려 있고, 끝나면 숨김과 클릭 통과가 되는 것을 확인합니다.
JS의 하드 타임아웃(25초)도 따로 있습니다	CSS 안전장치는 화면을 풀기만 합니다. JS는 25초가 지나면 정상적으로 열어서 뒤처리(`inert` 해제, 요소 제거)까지 합니다. 보통은 이쪽이 먼저 작동하고 CSS는 마지막 보험입니다.
로딩 중에는 뒤 화면을 `inert`로 막습니다	로딩 화면이 시각적으로만 덮고 있으면 Tab 키로 보이지 않는 뒤 화면의 버튼에 들어갈 수 있습니다. 로딩이 끝나면 우리가 막은 요소만 되돌립니다.
종류가 정해지기 전에는 문구를 숨깁니다	마크업의 기본 문구는 `OPENING PROJECT`라서, 새 프로젝트인데 첫 0.3초 동안 이 문구가 보이면 틀립니다. 시험에서 로딩 화면이 보인 시점(23ms)과 스크립트가 종류를 정한 시점(약 340ms)이 달라서 실제로 일어나는 일이었습니다.
`role="progressbar"`와 `aria-valuenow`를 씁니다	화면을 읽어 주는 도구가 "불러오는 중 39%"를 알 수 있게 합니다.
파형을 목업의 식 그대로 그립니다	목업과 같은 모양이어야 하고, 같은 입력은 늘 같은 모양이라 시험할 수 있습니다. 화면 너비에 맞춰 `viewBox`와 점의 수를 바꾸고, 창 크기가 바뀌면 다시 그립니다.
새 프로젝트의 파형은 더 잔잔합니다	목업의 `calm` 값(0.55)입니다. 불러올 것이 없는 짧은 로딩이라는 느낌을 줍니다.
시험 도구가 기본으로 로딩 화면을 끕니다	로딩 화면이 화면 전체를 덮으므로, 기존 시험이 버튼을 누르려 할 때 막힙니다. 시험 도구(`lib.mjs`)가 페이지가 열리기 전에 `window.__snxNoLoader = true`를 넣고, `createLoader`가 이 표시가 있으면 로딩 화면을 치우고 바로 열린 것처럼 동작합니다. 로딩 화면 시험(`launch(..., { loader: true })`)만 이 표시를 넣지 않습니다.

7. 시험

- `docs/handoff/s1-14_tests/loader_unit.mjs` (80건)는 브라우저 없이 확인합니다.
  - 종류: `projectId`가 없으면 `demo`, `?new=1`이면 `new`, 그 밖에는 `open`입니다. `new=0`, `new=true`는 `open`입니다.
  - 최소 시간: `open` 1600, `new` 1100, `demo` 800, 움직임 줄이기면 0입니다.
  - 단계 계획: 세 종류의 단계 글, 비중의 합이 1, 샘플 0개면 `NO SAMPLES`와 작은 비중, 샘플 수를 아직 모르면(`null`) `SAMPLES`입니다.
  - 진행률 합산: 비중 합산, 범위 밖 값 자르기, 0으로 나누지 않음, 숫자가 아닌 값을 0으로 봄입니다.
  - 부드럽게 따라가기: 목표를 넘지 않음, 줄어들지 않음, 최소 시간 한도, 음수 시간에도 던지지 않음입니다. 프레임을 모의로 돌려서, 최소 시간 1600ms 전에는 100%가 안 되고 1600~2400ms 안에 100%가 되며 값이 줄어들지 않는 것을 확인합니다.
  - 파형: 1440 너비에 점 241개, 같은 입력은 같은 모양, 모든 값이 0~120 안, `calm`이 작으면 진폭이 작음, 양 끝은 신호선으로 잦아듦입니다.
  - 정보 글(BPM, 박자, Key, 트랙 수를 점으로 이은 한 줄), 소수 BPM, 트랙 1개는 `TRACK`, 값이 없는 칸 건너뜀, 새 프로젝트는 `MASTER TRACK ONLY`입니다.
  - 샘플 모으기: 클립과 Sampler의 샘플을 중복 없이, `null`과 문자열 번호 처리입니다.
  - 소스 점검: 로딩 화면이 `<body>`의 첫 요소, `progressbar`와 `aria` 값, 세 단계, CSS 안전장치와 `z-index`, 움직임 줄이기, 최신 문법이 없음, `sentry-init.js`가 여전히 첫 `import`, 시험 도구의 표시입니다.

- `docs/handoff/s1-14_tests/ui_loader.mjs` (47건)는 실제 시퀀서를 열어 시간순으로 기록합니다. 샘플 받기를 느리게 하거나 실패시키려고 페이지가 열리기 전에 `fetch`를 감싸는 스크립트를 넣습니다. 자세한 내용은 315의 시험 절과 개요 90에 있습니다.
- 시험 중 발견한 일(1): 첫 실행에서 새 프로젝트와 예시 화면의 종류 문구가 `OPENING PROJECT`로 나오는 실패가 있었습니다. 로딩 화면이 보이는 시점(23ms)과 스크립트가 종류를 정하는 시점(약 340ms) 사이에 기본 문구가 비치는 실제 문제였고, CSS로 종류가 정해지기 전에는 문구와 단계를 숨기도록 고쳤습니다. 그 뒤 시험에서 "정하기 전 표본(44개)은 숨겨져 있고, 정한 뒤 표본(43개)은 보인다"를 확인합니다.